D2 Detailed Design

Member

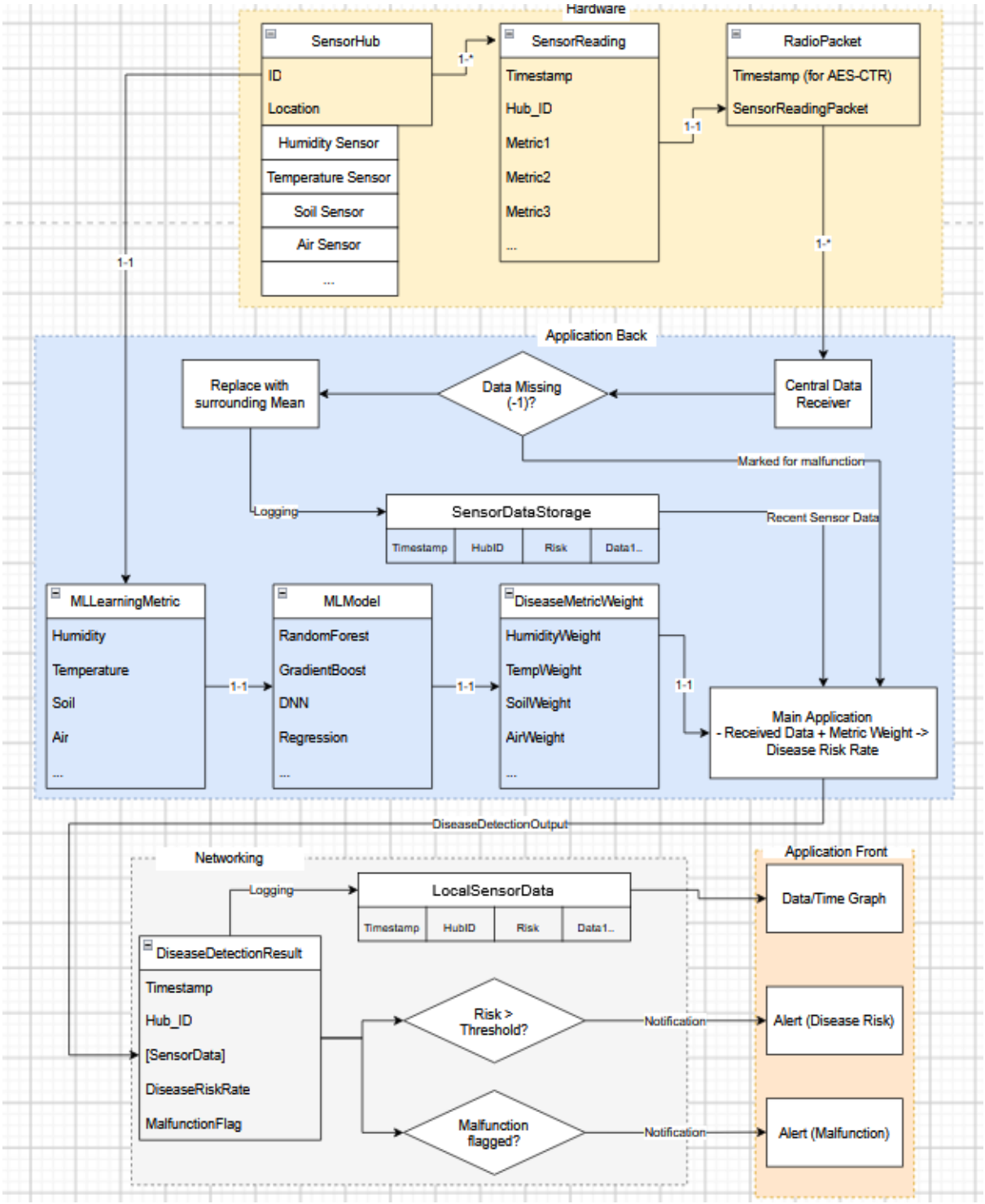
CS Senior: Chi Thanh Tran

EECE Senior: Luke Grochocki, Hebron Mekuria, Kian Palmer

Title, Goal, and Convention

- Project: A low-cost sensor system that include:
    - Sensor Hub: Control sensors that collect field data, read data using sensors, transmit it (using radio, or maybe another method) to a central receiver. ○ Central Application: Central Program connected to the central receiver that receive sensors data from Sensor Hub and decompress/decrypt said data. This program also hold the machine learning model and training data needed to check for disease.
    - Networking: Central Program, after doing its work, need to send data to "client application" to allow Farmer to access and check. In case where important information is needed to be shown (disease risk high, malfunction hardware, etc.), Central Program will also need to send these data to client application.
        - NOTE: Despite being "Network", our system are not planned to need a database/API, and in case needed, will be more decentralized, since this fit more to what a Farm/farmer usually set up
    - GUI: While farmer can check data on Central Application, a GUI for portable system like phone is likely preferred when they are far from the Farm. GUI receive data and information from Network to show said data to the farmer in a clean and interactive way that is easy to work with.

D2 Diagram


Hardware
SensorHub
ID
Location
Humidity Sensor
Temperature Sensor
Soil Sensor
Air Sensor
...
1-*
SensorReading
Timestamp
Hub_ID
Metric1
Metric2
Metric3
...
1-1
RadioPacket
Timestamp (for AES-CTR)
SensorReadingPacket
1-1
1-*
Application Back
Replace with surrounding Mean
Data Missing (-1)?
Central Data Receiver
Marked for malfunction
Logging
SensorDataStorage
Timestamp
HubID
Risk
Data1...
Recent Sensor Data
MLLearningMetric
Humidity
Temperature
Soil
Air
...
1-1
MLModel
RandomForest
GradientBoost
DNN
Regression
...
1-1
DiseaseMetricWeight
HumidityWeight
TempWeight
SoilWeight
AirWeight
...
1-1
Main Application
- Received Data + Metric Weight -> Disease Risk Rate
DiseaseDetectionOutput
Networking
Logging
LocalSensorData
Timestamp
HubID
Risk
Data1...
DiseaseDetectionResult
Timestamp
Hub_ID
[SensorData]
DiseaseRiskRate
MalfunctionFlag
Risk > Threshold?
Notification
Malfunction flagged?
Notification
Application Front
Data/Time Graph
Alert (Disease Risk)
Alert (Malfunction)

Design Rationale (Including Changes compared to D1):

- We chose a storage at the backend over the original local storage to accommodate for ML model requiring past data as INPUT. We missed this obvious need, but is important due to the need to measure the result through the change of metrics over time, rather than just one static data read. We accepted the trade-off of potential disconnection to user viewability because the main goal of detecting disease is prioritized and need to be improve over simple UX improvement.
- We chose usage of local storage over direct backend storage connection to avoid high dependency on what planned to be a largely private system. This is so that data can be received remotely and saved for viewing, but not as an instantaneous update. We accepted the trade-off of occasional temporary desynchronization is due to the inherent goal of the system to be used as hardware data research project, not as a heavy networking-based system, and this will save us time to focus on the main research focus
- We chose step-list system design over every other option due to the inherent goal of the project being for research. Since there is low/no need for a two-way communication between Client/Server, or a sensing cycle for changing metrics, the system follow a simple Input (Sensor Data over Time) -> Result (Disease Detected?). We accepted the trade-off of a more user-available system that can self-adapt because a focus in research-related result
-

Edge-Case

ID	Component	Category	Edge Case	Decision
1	Sensor	Bad Input	Sensor Data read nothing	Return as the Maximum possible value for said datatype
2	Central Data Receiver	Boundary Values	Data is abnormal	Changed said value to mean of 10 past

			(such as very large value intentionally created above)	value read of the same metric. Mark said data as malfunctioned
3	Central Data Receiver, Storage, Network	External Dependency	Network unreachable/ uncontactable	Retry 3 times, then stop. All components have their own timer to check for lateness, notify users

- Due to inherent hardware nature of the system, lost sensor data is unrecoverable.
- Since the system is also offline/local by most account, no concurrency should happen, and network issues are recoverable as long as connection can be recover.

Requirement Traceability Table

ID	Requirement	Element	Status
1	Save Sensor Data	SensorDataStorage	Covered
2	Show Recent Data as Graph	Network -> GUI	Covered
3	Alert when high risk of disease	Backend -> Network -> GUI	Covered
4	Alert on data missing	Backend -> Network -> GUI	Covered
5	Detect failed sensors	Sensors change reading to noticeable value on fail	Covered
6	Clean missing data	Replace missing data with mean of past data	Conflict(?)
7	Failed Networking	Local timer check for late packet -> Network -> GUI	Covered

Peer Review:

Maxwell Blevins - blevinmw@mail.uc.edu

- I would expand more upon your idea of what your core algorithms are. Right now they come across as vague.
- You should explain more why the ML Model that you are using is going to work even though you aren't building it. Specificy the model and give some sort of blurb to say why it does the trick.
- In place of the API contract I would at least show how your data is going to flow from one part to another. Explain what the input is going to look like and what the output is going to look like at the very least.
- It could be worth saying what specific big libraries you are going to use with Python sense it's the standard for a lot of ML projects like this one.

Anonymous

- One suggestion would be to explain more about how the system will detect disease risks and how accurate the detection needs to be

Issue	Raiser	Action	Priority
Expand Core Algorithm	Maxwell Blevins	Explain more clearly the overall code and loops of the core system, avoid overestimating potential viewer, go deeper on reasoning for the choice	Must-fix
Expand on ML Model Choices	Maxwell Blevins	Choose and explain the choice of ML Model, why the model specifically allow the project to work, and other alternative (or why not alternative)	Must-fix
More specific data flow	Maxwell Blevins	A more specific data flow that show how data go from the sensor hub to the end application. This should be simple due to the project's research-orientated view, but might change often depend on future research on needed metric. A base flow can be built first that can be update when needed	Consider
Expand on general Library usage	Maxwell Blevins	Show all the library that needed to be use for every code used. Since this might change and depend heavily on what will	Consider

		finally be done at the end, it might be difficult to have a clear list, but worth making a base on guarantee-to-use library	
Overall explanation on detection	Anonymous	A more specific explanation on how the detection go is somewhat the plan of the research, and while we have the broad idea of how it will go, a complete explanation will need to be formed through-out the research	Out of Scope for now
Expand on detection accuracy	Anonymous	Similar to above, the accuracy of detecting disease using sensor data is the large part of the research, and it might be difficult to just “plan out” one that work perfectly	Out of Scope for now